SISTEMA DE GESTIÓN ACADÉMICA – DECISIONES DE DISEÑO

YEIMMY JOHANA JIMENEZ YAYA

UNIVERSIDAD DE MANIZALES
FACULTAD DE CIENCIAS E INGENIERÍA
2026

1. Introducción

En este documento explico las decisiones que tomé para mejorar mi Sistema de Gestión Académica. En esta actividad hice que Persona fuera abstracta, agregué los grupos con su horario y creé excepciones para controlar errores.

2. Descripción general

El sistema maneja personas (estudiantes, docentes y administrativos), programas, asignaturas y grupos. Cada grupo tiene una asignatura, un docente, un horario y estudiantes, con un cupo máximo. Está hecho en Python con programación orientada a objetos y organizado en models/ (las clases), controllers/ (la búsqueda de programas y asignaturas) y main.py (las pruebas).

3. Desarrollo

3.1 La clase abstracta Persona

En el sistema nunca hay una persona sola: siempre es un estudiante, un docente o un administrativo. Por eso Persona es una clase abstracta (hereda de ABC). Solo guarda lo que tienen en común, que es la identificación, el nombre y el correo, y no se pueden crear objetos de ella. Si se intenta, Python lanza un TypeError, y en main.py lo capturo con try/except para que el programa siga**.**

3.2 El Método abstracto y las clases derivadas

realizar_actividad_principal() está declarado como abstracto en Persona, porque lo que hace cada persona depende de su tipo. Estudiante lo implementa mostrando el programa que cursa y su semestre, Docente mostrando la facultad donde da clases, y Administrativo mostrando su cargo y su dependencia. Si una de estas clases no lo implementara, Python no dejaría crear objetos de ella.

3.3 Relación entre las clases del grupo

GrupoAcademico recibe objetos ya creados de Asignatura y Docente, no solo sus nombres, y por eso puede pedirles datos, por ejemplo get_nombre(). También guarda una lista de objetos Estudiante, que se agregan con

matricular_estudiante() y se quitan con retirar_estudiante(). Estas tres clases existen por su cuenta, aunque el grupo desaparezca. En cambio, el Horario lo crea el propio grupo dentro de su constructor

3.4 3.4 Composición entre el grupo y el horario

GrupoAcademico crea un Horario en su constructor y no hereda de él. Un grupo tiene un horario, pero no es un horario. Además, el horario solo sirve dentro de un grupo, porque describe cuándo y dónde se dicta ese grupo, por eso lo crea el grupo. La herencia se usa cuando algo es un tipo de otra cosa, como un estudiante que es una persona.

3.5 Excepciones personalizadas

Todas las excepciones heredan de SistemaAcademicoError y están en excepciones.py:

- **EstudianteYaMatriculadoError**: se lanza en matricular_estudiante() cuando el estudiante ya está en la lista del grupo.
- **GrupoLlenoError:** se lanza en matricular_estudiante() cuando ya no hay cupo.
- **EstudianteNoMatriculadoError:** se lanza en retirar_estudiante() cuando el código no aparece en el grupo.
- **AsignaturaNoEncontradaError:** se lanza en buscar_asignatura() de GestorAcademico cuando el código no existe.
- **ProgramaNoEncontradoError**: se lanza en buscar_programa() de GestorAcademico cuando el código no existe.

En main.py cada una se captura con try/except, se muestra el mensaje y el programa sigue funcionando.

3.6 Polimorfismo

El polimorfismo de la Actividad 1 se mantiene. En main.py guardo estudiantes, docentes y administrativos en una sola lista y la recorro con un for que llama a mostrar_informacion() y realizar_actividad_principal(). Aunque ahora Persona es abstracta, el funcionamiento es igual: cada objeto ejecuta su propia versión del método, sin usar if para preguntar el tipo.

4. Conclusiones

Aprendí que una clase abstracta obliga a las clases hijas a tener ciertos métodos, y que la composición se usa cuando una clase tiene un objeto de otra. También vi que las excepciones ayudan a que los errores se entiendan y el programa no se caiga.

5. Declaración de uso de IA

Usé Claude (una herramienta de IA) como apoyo en las dos actividades para entender conceptos de POO, armar la base del código, organizar este documento y resolver errores de Git. Revisé y ejecuté el código, y cambié mensajes y datos para dejarlos a mi manera.